

# LumiLeaf Production Deployment Handbook

## 1. Purpose

Guide for deploying LumiLeaf from a developer laptop to a production environment used by factory staff.

## 2. Current Development Workflow

Developer PC: IntelliJ IDEA + XAMPP (Apache/MySQL) + Spring Boot. Access via localhost:8080. Suitable only for development.

## 3. Production Architecture

Employees -> HTTPS Domain -> Nginx -> Spring Boot Application -> MySQL Database. Application runs as a Linux service; no IntelliJ or XAMPP.

## 4. Infrastructure Checklist

Ubuntu Server 24.04 LTS, Java 21, MySQL 8/MariaDB, Nginx, SSL certificate, domain, UPS, automated backups, firewall, SSH access.

## 5. Folder Structure

/opt/lumileaf/app (JAR)  
/opt/lumileaf/uploads (reports, QR, images)  
/opt/lumileaf/config  
/opt/lumileaf/logs  
/opt/lumileaf/backups

## 6. Database

Separate MySQL server/database. Never package data inside the application. Backup daily and test restores.

## 7. First Deployment

Build with Maven, upload JAR, configure application-prod.properties, start via systemd, configure Nginx and HTTPS, verify application.

## 8. Updating Production

Develop locally, commit to Git, test in staging, backup DB, replace JAR, restart service, run Flyway/Liquibase migrations if schema changes, verify.

## 9. Zero-Downtime

Use blue-green or rolling deployments when continuous availability is required. Small internal systems may accept a short restart window.

## 10. Rollback

Keep previous JAR, backup database before migration, revert to previous version if validation fails.

## 11. Security

HTTPS, strong passwords, RBAC, audit logs, regular OS updates, least-privilege DB accounts.

## 12. Monitoring

Monitor CPU, RAM, disk, application logs, database health, backups, SSL expiry.

## 13. CI/CD

GitHub -> Build -> Automated tests -> Package JAR -> Deploy to staging -> Approve -> Production.

## **14. Operational Workflow**

Developer -> Git -> Build -> Test -> Backup -> Deploy -> Migrate -> Verify -> Users continue working.

## **15. Future Improvements**

Docker, Kubernetes, centralized logging, Prometheus/Grafana, high availability database, object storage for uploads.